Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version 13 October, 2026.

Digital Object Identifier 10.1109/XXXX.2026.1234567

Balancing Security and Usability: Adaptive CAPTCHA Orchestration via Reinforcement Learning

Anonymous Author(s)

[1]Affiliation Omitted for Double-Anonymized Review

ABSTRACT Websites increasingly receive a continuous bot-likelihood score for each user instead of a pass/fail decision, but turning that score into an action (no challenge, a CAPTCHA of some difficulty, or a block) is still left to hand-written rules. We study whether reinforcement learning (RL) can learn this mapping over an 11-level action ladder. A small LSTM *Humanity Scorer* reads mouse dynamics (displacement, inter-event time, speed and acceleration); its calibrated output, a running session score, challenge history and the number of active users form the state of a DQN, PPO, Neural-LinUCB or Neural Thompson Sampling controller. All evaluation is simulated: bots solve challenges stochastically (an item-response model), and human pass rates, solve times and abandonment are taken from published CAPTCHA user studies, so usability is *modelled, not measured*. On a held-out block of recordings never used for scorer or policy training, DQN reaches a Discrimination Index (humans kept % − bots through %) of 98.9 (95% CI ±0.4 over 5 training seeds) against 78.7 for the best static threshold policy; removing the Humanity Score drops it to 23.6. When challenges are solved stochastically, Thompson Sampling matches or exceeds DQN (91.5 vs 88.6). We also report the limits of the approach: rewards computed only from observable challenge outcomes collapse to "never challenge", and when the scorer is miscalibrated (covert bots or bot-like humans), frozen RL policies degrade faster than static thresholds, while online training recovers only when its reward does not inherit the scorer's error.

INDEX TERMS Behavioural biometrics, bot detection, CAPTCHA, calibration, contextual bandits, mouse dynamics, reinforcement learning, web security.

I. INTRODUCTION

AUTOMATED traffic is now close to half of all web requests, and 37% of 2024 traffic was attributed to malicious bots [1]. Bots scrape content [2], post spam [3] and stuff credentials [4]. The standard defence is the CAPTCHA [5], [6], but every challenge also costs legitimate users time and some of them leave: in a large field study, humans solved distorted-text CAPTCHAs correctly only 71–98% of the time depending on the scheme [7], and in a recent study of modern CAPTCHAs a substantial share of participants abandoned the task when a challenge was presented [8]. At the same time, vision-language models now solve many visual challenges [9], [10], and reCAPTCHA v2 has been broken with off-the-shelf detectors [11].

Risk-based services such as reCAPTCHA v3 therefore return a continuous score rather than a decision [12]. That moves the hard part to the website operator, who must decide what to do with a score of, say, 0.6. Three things make this decision hard in practice. *(i) One threshold does not fit every context*: a score that justifies frictionless browsing may be inadequate for a payment page [13], and fixed thresholds become stale as adversaries adapt [14], [15]. *(ii) Scores are not comparable across providers*: each detector has its own undisclosed calibration, so combining them is ad hoc [16]. *(iii) The trade-off is sequential*: a mild first challenge can be escalated if the user still looks suspicious, and a policy that challenges early can relax later. Rule-based web application firewalls [17] encode these choices by hand.

This paper asks whether an RL controller can learn the score-to-action mapping, and what can and cannot be concluded from a simulation-based evaluation of it. Our contributions are:

i) **A graduated orchestration formulation.** The controller chooses one of 11 levels (0: no challenge; 1–9: increasingly strong challenges; 10: block) every 10 s of a session, using a behavioural score and session and server context (Section III).

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

TABLE 1. Scope of the Evaluation

Status	Component
Evaluated	LSTM scorer on our recordings; RL and static policies in simulation; 11 abstract levels with literature-based human/bot outcome models; bot volumes 20–1000; scorer-calibration drift with external data
Implemented, not evaluated	Playground website and telemetry collection; web SDK
Proposed only	Mobile SDKs, device fingerprinting, routing to real CAPTCHA providers, server-load (DoS) features, adaptive/VLM attackers, user study

ii) **A compact, calibrated behavioural scorer** (6,081 parameters, temperature-scaled), chosen by grouped cross-validation, together with the data protocol needed to avoid leakage between scorer, policy and evaluation (Section B).

iii) **A literature-grounded simulator** in which bots solve challenges with a probability that depends on their capability, and human pass rates, solve times and abandonment rates come from published user studies [7], [8] (Section F).

iv) **An evaluation with confidence intervals** of DQN, PPO, Neural-LinUCB and Neural Thompson Sampling against static threshold policies and an ablation without the score, on held-out recordings over 5 training seeds (Section IV).

v) **Negative results that bound deployment**: label-free rewards, and the behaviour of offline and online-trained policies when the scorer's calibration drifts (Sections D–E).

Scope. Table 1 separates what was evaluated from what is only proposed. In particular, no real CAPTCHA provider, user study, server-load model or adaptive attacker is part of the evaluation; we therefore make no claim about denial-of-service mitigation in the sense of server resources, only about robustness to the volume and composition of bot traffic.

II. RELATED WORK

CAPTCHA security and usability. CAPTCHAs were introduced as hard AI problems [5] and have been surveyed extensively [6], [18]. Their usability cost is well documented: Yan and El Ahmad analysed design factors [19], Bursztein et al. measured human accuracy and solve times on 318,000 CAPTCHAs from 21 schemes [7], and Searles et al. measured solve times, preferences and abandonment for modern challenges with 1,400 participants [8]. On the attack side, text CAPTCHAs have generic solvers [20], reCAPTCHA has been bypassed with audio and image models [11], [15], its v3 score has been gamed with RL [21], and multimodal models solve visual-reasoning challenges [9], [10], [22].

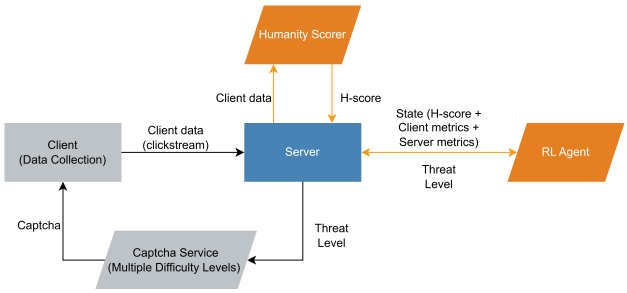


FIGURE 1. Modular architecture. Tiers 1–3 are implemented; tier 4 is simulated in this paper (Table 1).

Behavioural biometrics. Mouse dynamics separate humans from bots and individuals from each other [23]–[25]. Human motion follows Fitts' law with motor noise [26], while bots range from straight-line movers to Bézier-curve and learned trajectory generators [27]–[29]. Iliou et al. combined web logs and mouse biometrics to detect advanced web bots [30]; Acien et al. did so with touch and sensor data [31]. Recurrent networks are a common choice for such sequences [32], [33].

Adaptive challenges and RL. Aura-CAPTCHA uses RL and GANs to adapt challenge difficulty [34]; it does not orchestrate a graded ladder ending in a block. Contextual bandits choose an action from features to maximise immediate reward [35], [36]; DQN [37] and PPO [38] optimise discounted return in a Markov decision process [39], and RL has been applied to fraud detection [40] and anomaly detection [41]. Publicly available labelled human/bot interaction data remain scarce [42], which is why we report data sizes and leakage controls in detail.

III. METHODOLOGY

A. System Architecture

Fig. 1 shows the four tiers. *Tier 1* (client telemetry) collects cursor coordinates and timestamps in the browser and normalises them to a platform-independent JSON format; only the web version exists. *Tier 2* (Humanity Scorer) converts each 10-second chunk of telemetry into a bot probability; any scorer with a probabilistic output, including a commercial one, can be substituted. *Tier 3* (RL controller) maps the score and context to a level. *Tier 4* (fulfilment) would route each level to a concrete challenge; in this paper levels are simulated (Section F), not served by real providers.

B. Data Collection

A playground blog website (navigation, form filling, commenting) recorded `mousemove` events (x, y, t) with a minimum gap of 10 ms. Each session lasted about 120 s. Four bot generators of increasing sophistication interacted with the same site: *naive* bots that jump directly to targets, *human-like* bots that move in straight lines at constant speed, *mimic* bots that follow Bézier curves with varying speed [29], and *fallible* bots that add overshoots, hesitations and corrections. In total we have **156 recordings**: 44 human and

112 bot recordings, 42,913 movements, from two collection campaigns three months apart. The 44 human recordings come from **3 participants**.

Ethics. The study followed established ethical guidelines for research with human participants. Participants gave informed consent, and only anonymous cursor trajectories were stored; no personally identifiable information was retained.

Partition (answers R1.2). The split is made *at the recording level, before any windowing or augmentation*, so no window of a recording appears in two blocks. Recordings are divided into a scorer block, an RL block and a held-out *evaluation block* using rotating folds; the results in this paper use fold 2, whose evaluation block has 14 human and 36 bot recordings and is never used to fit or select the scorer or the policy. Within the scorer block we cross-fit five scorers so that every RL training session is scored by a model that did not see it. Every opening of the evaluation block is logged. No augmentation is used (Section C). With three participants, the human data cannot support claims about the general population; we report a leave-one-campaign-out check as the closest proxy for unseen users.

C. Humanity Scorer

Input. A chunk (the movements in one 10 s step) is converted to per-event features $[\Delta x, \Delta y, \Delta t, v, a]$: displacement, inter-event time, speed $v = \|(\Delta x, \Delta y)\|/\Delta t$ and its first difference a. Absolute screen position is deliberately not used, because it encodes page layout rather than behaviour. Features are standardised with training-set statistics and cut into windows of 32 events; the last window is padded and a sixth binary channel marks padded steps (Fig. 2).

Model. One LSTM layer [32] with 32 units reads the window; its final hidden state passes through a 32-unit ReLU layer and a sigmoid output (6,081 parameters). Window probabilities are temperature-scaled [43] ($T = 1.45$) and averaged over the chunk, giving $q = P(\text{bot})$; the Humanity Score is $H = 1 - q$. Training uses binary cross-entropy, Adam (learning rate 3×10^{-3}), dropout 0.25, recurrent dropout 0.2 and early stopping on validation loss.

Why this design. The original submission used two LSTM layers (200 and 100 units, 304,049 parameters) on absolute (x, y) coordinates. A grouped 5-fold search over architectures and input representations on all 156 recordings showed that the original model had about 1,170 parameters per training window, that a single 32-unit layer on relative motion was better on every criterion (cross-validated session AUC 0.977 vs 0.906; leave-one-campaign-out AUC 0.890 vs 0.842), and that Gaussian or jitter augmentation did not help a correctly sized model. Adding timing on top of displacement changed AUC by $+0.007$ on average, with inconsistent sign, so we make no claim that timing features are essential; we keep them because acceleration-type features are standard in mouse-dynamics work [23].

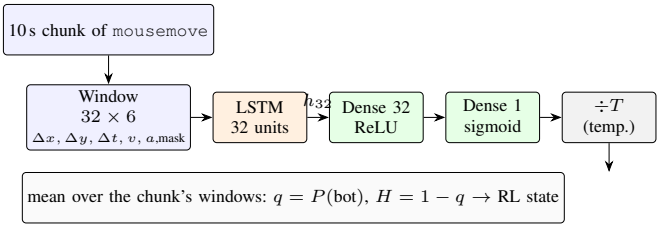


FIGURE 2. Humanity Scorer. Each 10 s chunk is cut into 32-event windows; the window probabilities are temperature-scaled and averaged into the chunk score q.

D. RL Formulation

Each user session is an episode of up to 12 decision steps (one per 10 s chunk). Many users are active at once; the controller acts for each active user at each step.

State. The five features, all in $[0, 1]$, are: (1) the current chunk score q_t; (2) the running mean of q over the session; (3) the number of challenges the user has passed, $\tanh(n/5)$; (4) the last level shown, divided by 10; and (5) the number of active users, scaled. Features (3)–(5) need no scorer.

Actions. Level $T \in \{0, \ldots, 10\}$. Level 0 shows nothing; levels 1–2 are invisible to the user (passive score, honeypot); levels 3–9 are visible challenges of increasing difficulty (checkbox, easy and standard text, image grid, hard text, multi-round, interactive game); level 10 blocks the session (Table 2).

Reward (training only). For a bot of capability B_s: if it is stopped, $r = R_{\max} - c_o \max(0, T - B_s)$, which discourages using a stronger level than needed; if it passes, $r = R_{\text{leak}} - c_u \max(0, B_s - T)$. For a human: $r = R_{\max}$ at $T = 0$; $r = R_{\max}/2 - c_f\, s_T$ after passing a challenge that took s_T seconds; and $r = R_{\text{FP}} - c_f\, s_T$ if the human fails (a false positive). Friction is priced in measured seconds rather than with the exponential β^T of the original submission, whose value at $T = 10$ (-999) dominated every other term (R1.3). This reward requires the user's true class, which the simulator knows and a deployed system does not. Section D tests rewards that a deployment could compute.

Worked example. Consider a fallible bot of capability $B_s = 4$ whose second chunk scores $q = 0.82$, with session mean 0.75, no challenges passed, last level 0 and 400 active users. The state is $(0.82, 0.75, 0, 0, 0.67)$. Suppose the policy chooses $T = 6$ (image grid). The bot passes with probability $\sigma(1.5(4.5 - 6)) = 0.095$. If stopped, the session ends and $r = 50 - 2.5 \cdot 2 = 45$; if it passes, $r = -150$ and its next state records one passed challenge and last level 0.6. A human shown the same level passes with probability 0.86 in a median 9.7 s, giving $r = 25 - 2 \cdot 9.7 = 5.6$, or fails and gives $r = -100 - 19.4$. A typical trained DQN shows nothing to a user with $q < 0.2$ and escalates through levels 3–7 as q and the running mean rise.

E. Learning Algorithms

DQN [37] approximates $Q(s, a)$ with a 128–64 ReLU network and minimises the Huber TD loss to the target

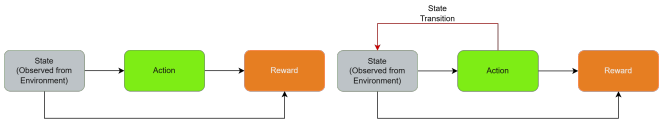


FIGURE 3. Contextual bandit (left) and full RL (right) interaction loops.

$r+\gamma \max_{a'} Q_{\theta^-}(s',a')$ with ϵ-greedy exploration. Its replay memory is split into a human and a bot buffer (10,000 transitions each) and every mini-batch draws half from each. *This balancing uses the true class*, as the oracle reward does; for the label-free rewards in Section D we balance on the observable outcome (challenge failed vs not) instead.

PPO [38] uses the same 128–64 body with separate policy (11 logits) and value heads, the clipped surrogate objective, GAE, and one update per simulated episode. Its architecture was chosen by a search over five bodies (Table 2).

Neural-LinUCB and Neural Thompson Sampling [35], [36] learn an embedding $\phi(s)$ with an MLP and keep, per action, ridge statistics $A_a = \lambda I + \sum \phi\phi^\top$, $b_a = \sum r\phi$. LinUCB picks $\arg\max_a \phi^\top \hat{\theta}_a + \alpha\sqrt{\phi^\top A_a^{-1}\phi}$ with $\hat{\theta}_a = A_a^{-1}b_a$; Thompson Sampling samples $\tilde{\theta}_a$ from the posterior and picks $\arg\max_a \phi^\top \tilde{\theta}_a$. A single Gaussian posterior was consistently worse than a mixture of Gaussians, so Thompson Sampling uses a 2- or 3-component mixture. *Moving feature space (R1.6):* because ϕ changes when the network is updated, statistics accumulated under old embeddings become inconsistent. At the end of training we therefore recompute every A_a and b_a from the stored transitions in the final embedding space; the statistics used at evaluation always match the network that produces ϕ.

Fig. 3 contrasts the two families: bandits treat each decision independently, while DQN and PPO propagate value across the steps of a session.

F. Simulated Environment

Traffic. A training episode samples 50–250 humans and 0–1000 bots, drawing each simulated user's mouse data from a recording in the RL block and each bot's capability B_s uniformly from $\{0,\ldots,9\}$. Because human and bot counts vary independently, the number of active users is not a proxy for the bot count (R1.5).

Deterministic world (original submission). A bot is stopped iff $T > B_s$, and humans always pass. It is the $\alpha \to \infty$ limit of the model below and is kept for comparison.

Stochastic ("grounded") world (R1.1). A bot passes level T with an item-response probability $\sigma(\alpha(B_s+0.5-T))$, $\alpha = 1.5$, so strong bots sometimes solve hard challenges and weak bots sometimes solve easy ones. Human pass rates and solve times per level are taken from Bursztein et al. [7] and Searles et al. [8] (Table 2); a human who fails is lost. After a pass a human may leave with probability $1-e^{-s_T/45\,\mathrm{s}}$. This last term is an *assumption*: we found no study measuring abandonment after a successful solve.

TABLE 2. Parameters and Their Sources (R2.4). Lit. = Published Study; Search = Chosen by Grouped CV on the RL Block; Assum. = Modelling Choice

Parameter	Value	Source
Environment		
Human pass rate, median s, levels 3–8	.98/3.7 ….70/13	Lit. [7], [8]
Level 9 (game) / 10	.62/25 s / block	Lit. [8]/Assum.
Bot solve slope α; B_s	1.5; $\mathcal{U}\{0..9\}$	Assum. (sweep 0.5–∞)
Leave after pass, τ	45 s	Assum. (sweep 22–80)
Abandon at presentation (v2)	2.6–9.2%	Lit. [8]
Reward		
$R_{\max}$, R_{leak}, R_{FP}	50, -150, -100	Assum. (9-setting sweep)
c_o, c_u, c_f	2.5, 5, 2/s	Assum. (sweep)
Learners		
DQN/PPO body	128–64 ReLU	Search (6 / 5 bodies)
LinUCB / TS embedding	64–32 / 128–64–64	Search
TS posterior	MoG (2–3 comp.)	Search
Episodes; steps/episode	200; $\leq$12 per user	Assum.
DQN: γ, lr, ϵ decay, target sync	.95, 5×10^{-4}, .9995, 200	Standard [37]
PPO: clip, GAE λ, epochs, lr	.2, .95, 10, 3×10^{-4}	Standard [38]
LinUCB α, ridge λ	1, 1	Standard [35]
Seeds	5 train × 50 eval	—

Abandonment-at-presentation world (v2, sensitivity). Replacing that assumption, a human may abandon when a visible challenge is shown (and again before a retry), never after passing, with per-type rates from Searles et al. [8] (2.6–9.2% for levels 3–9, scaled by their in-context/pooled ratio of 1.34) and a position multiplier that front-loads abandonment ($\times3.25$ for the first challenge, $\times0.55$ from the fourth). An abandoning human is charged R_{FP}, consistent with the metrics, and level 10 blocks everyone. Used only in the pilot of Section E.

TABLE 3. Confusion Matrix of the Original Scorer on Its Test Set

	Predicted human	Predicted bot
Actual human	163	11
Actual bot	16	93

G. Evaluation Protocol and Metrics

Each trained policy is evaluated on traffic built from the evaluation block, with 100 humans and $\{20, 100, 200, 500, 1000\}$ bots, 50 evaluation seeds per volume, for each of 5 training seeds. Let S_H and S_B be the fractions of humans and bots that complete their session. We report the **Discrimination Index** $\mathrm{DI} = 100(S_H - S_B)$, the **Balanced Orchestration Score** $\mathrm{BOS} = 2S_H K_B/(S_H + K_B)$ with $K_B = 1 - S_B$ (the F1-style harmonic mean [44]), and the scale-invariant SI-F1, the F1 of S_H with normalised precision $S_H/(S_H + S_B)$, plus mean friction in seconds. *Zero-bot traffic is excluded* from every metric and average (R1.4): S_B is undefined without bots, and the original table filled it with conventions that were not stated. Uncertainty is reported *across training seeds* (each seed averaged over volumes and evaluation seeds), as a 95% t-interval; comparisons use Welch's test with Holm correction [45]. We use "significant" only for such tests.

IV. RESULTS

A. Humanity Scorer

Original model. On the original temporal hold-out (283 test windows from a later campaign) the original two-layer model reached 90.46% accuracy, 93.67% recall and ROC-AUC 0.951 (Table 3).

Revised model. The five cross-fitted scorers used for RL training reach validation AUC 0.950–0.986 and held-out AUC 0.920–0.992. On a pool of unseen sessions, the selected architecture has window AUC 0.954 and session AUC 0.971; temperature scaling reduces the expected calibration error from 0.064 to 0.046, with Brier score 0.065 (R1.7). Leave-one-campaign-out (unseen participant sitting and hardware) lowers session AUC to 0.890, which is the more honest estimate for new users. Fallible bots are the hardest family (AUC 0.918 against humans), human-like bots the easiest (0.991).

External data (R1.9). The original submission reported near-perfect fine-tuned results on the web bot detection dataset of Iliou et al. [30] and 95.18% on Balabit [46]. On re-examination these numbers do not support a bot-detection claim and we withdraw them: Balabit contains only human users, so its accuracy measured only how many human windows were not flagged, and the web-bot runs split *windows* at random (windows of one session fell on both sides) and used the test set for early stopping. In this revision external data are used only as *unseen populations*, with a session-level split for the web-bot data (advanced bots and humans) and a user-level split for Balabit (5 users for injection, 5 held out). Without fine-tuning the scorer does not transfer: advanced web bots receive a mean $P(\text{bot})$ of only 0.30, and Balabit humans are ranked as more bot-like than those bots (AUC 0.13). This motivates the drift study of Section E.

B. Main Comparison on Held-Out Recordings

Table 4 replaces Table 4 of the original submission. The original table had two problems: the RL policies were trained and evaluated on the scorer's own test recordings, and its "±" measured variation across bot volumes rather than across training runs. Here every number comes from the held-out evaluation block, and intervals are across 5 training seeds.

DQN improves on the best static policy by about 20 DI at every seed (worst seed 98.6 vs 78.7) and keeps all humans up to 200 bots, 95% at 1000. Removing the Humanity Score from the state costs 75 DI: without it the agent cannot tell users apart and lets 64% of bots through. Thompson Sampling has the least friction (1.7 s) but is seed-sensitive (worst seed 73.4), and LinUCB is unstable (13–93). When the numbers of humans (64–239) and bots (3–717) are both varied, drawn independently for each run, the results barely change (DQN 99.7, best static 78.1; Table 4), so the comparison does not depend on a fixed human population. The two counts are drawn per run rather than crossed in a full factorial grid. Two caveats bound these numbers. First, the evaluation block is small (14 human and 36 bot recordings) and the scorer separates it almost perfectly, so DQN is near a ceiling; on the validation folds used for architecture search the same DQN scores 83.6 ± 0.8 (SE). Second, leakage of training recordings into evaluation would inflate scores: in a role-reversal test (two agents, each evaluated on the recordings it trained on and on those it did not), DQN scores 4.5 DI higher on seen recordings (SE 2.1, 10 splits), while the policies that do not train show a gap of exactly zero. The ranking is unchanged on unseen recordings (DQN 78.2 vs 66.5 for the best static).

Population confound (R1.5). Freezing the active-user feature at any constant from 25 to 800 changes human survival by at most 4.4 points and bot survival by at most 1.8, so security outcomes do not depend on reading the attack size from the population. The *share of users challenged* does depend on it (0.305 unfrozen; 0.204–0.357 frozen), so this feature mainly controls how cautious the policy is.

C. Stochastic Challenge Outcomes

Table 5 repeats the comparison in the grounded world, where challenges are passed probabilistically and humans can fail or leave. Policies retrained in this world are compared with the deterministic-world DQN transferred without retraining.

With stochastic outcomes the advantage of DQN over Thompson Sampling disappears: Thompson Sampling has the highest DI at every bot-solve slope tested (α from 0.5 to 12: 90.3–92.2 vs 86.1–86.9 for DQN), mainly because it challenges less and loses fewer humans to abandonment (3.2% vs 10.6%). Learned policies still beat static thresholds.

TABLE 4. Held-Out Evaluation (Deterministic World, Fold 2). Mean over Bot Volumes 20–1000 (100 Humans) and 50 Evaluation Seeds; Varying Traffic: Humans 64–239 and Bots 3–717 Drawn per Run; ± Is the 95% CI over 5 Training Seeds; Static Policies Are Deterministic Given the Scorer

	DI by bot volume											
Policy	20	100	200	500	1000	**DI**	**Worst seed**	**DI, varying traffic**	**BOS**	**SI-F1**	**Humans kept**	**Friction (s)**
DQN	100.0	100.0	100.0	99.3	95.1	**98.9** ± 0.4	98.6	**99.7** ± 0.1	0.994	0.994	0.989	7.0
Thompson Sampling (MoG3)	84.7	90.7	95.0	92.0	92.3	90.9 ± 12.4	73.4	88.9 ± 17.6	0.953	0.954	0.945	1.7
LinUCB	55.4	53.8	55.1	51.4	42.0	51.5 ± 36.5	13.1	49.6 ± 36.1	0.641	0.641	0.549	33.0
Static Multi-Threshold	80.0	78.6	78.8	78.2	78.0	78.7	—	78.1	0.893	0.893	0.887	9.4
Static Single-Threshold	77.7	77.7	77.7	77.7	77.7	77.7	—	77.6	0.874	0.874	0.777	6.7
DQN without Humanity Score	24.8	24.1	20.2	30.5	18.3	23.6 ± 10.8	13.9	21.5 ± 11.1	0.384	0.665	0.880	29.5

TABLE 5. Grounded Stochastic World ($\alpha = 1.5$), Mean over 5 Training Seeds

Policy	**DI**	**Humans**	**Bots**	**Aband.**	**Fric. (s)**
Thompson Sampling (MoG3)	**91.5**	0.956	0.041	0.032	2.6
DQN (retrained)	88.6	0.886	0.000	0.106	5.8
DQN (deterministic-trained)	86.4	0.864	0.000	0.134	6.6
Static Single-Threshold	82.6	0.826	0.000	0.116	11.5
Static Multi-Threshold	80.6	0.813	0.007	0.164	9.4
LinUCB	24.3	0.272	0.029	0.593	40.8
DQN without Humanity Score	7.5	0.512	0.437	0.443	24.9

The retrained DQN's mean includes one collapsed seed (49.8), which is why its seed SD is 21.7.

Reward and abandonment sensitivity (R1.3). Retraining DQN under nine reward settings (leak penalty, friction form and scale, over- and under-kill coefficients; 5 seeds each) gives a policy that beats every static baseline on the seed mean in all nine; 3 of the 45 runs collapsed. Under the shortest leave-after-pass constant ($\tau = 22$ s) the fixed policies rank DQN 91.7, Static Single 81.4, Thompson Sampling 75.9, Static Multi 64.8, LinUCB 1.8.

D. Rewards a Deployment Could Compute

A deployed controller sees only observable outcomes: a session that failed a challenge and ended may be a blocked bot or a lost human; one that passed may be a satisfied human or a leaked bot. In our simulation 47.5% of "passed and continued" events are bots and 11.6% of "failed and ended" events are humans. We trained DQN with 12 rewards computed from such outcomes (different penalties, potential-based shaping, a learned reward model, delayed abuse labels): *all 12 converged to never challenging anyone* (DI ≈ 0). Two scorer-based alternatives work in the stationary setting: a *score-only* reward that replaces the class label with the score (DI 85.3, seed SD 8.3), and a *posterior* reward that weights the oracle reward by the posterior class probability, using an EM estimate of the current bot share (DI 92.8, SD 5.9; oracle 88.6). Neither difference from the oracle is significant ($p = 0.70$ and $p = 0.15$ before correction). The posterior reward uses the simulator's own outcome model and is therefore an upper bound for this idea, and both rewards rely on the scorer being right, which the next section tests.

E. Calibration Drift: Offline vs Online Training

We simulate a scorer whose calibration degrades. In the *bots-look-human* direction a fraction ρ of evaluation bots is replaced by advanced web bots from an external dataset [30], which the scorer rates as human-like; in the *humans-look-bot-like* direction a fraction ρ of humans is replaced by Balabit users [46]. As ρ goes from 0 to 1 the calibration error of the shifted class grows from 0.07 to 0.61 (bots) and from 0.13 to 0.45 (humans), while session-level AUC stays ≥ 0.99, i.e., the ranking of sessions barely changes but the score's probabilities become wrong. We compare (a) policies trained offline and evaluated frozen, and (b) DQN and PPO trained online for 300 episodes, with the shift injected from episode 100, against the same protocol without the shift (control). The scorer is a cross-fitted seed chosen because it ranks the external bots best; this choice was made after inspecting the external data.

Offline-trained, frozen (Fig. 4, left). All score-using policies degrade as calibration error grows; DQN without the score stays flat at about 16, as expected for a negative control. When bots look human, the RL policies fall *further* than static thresholds (at $\rho = 1$: DI 10.2 DQN, 3.8 PPO vs 28.4 and 39.5 static), because they rely on the score more. When humans look bot-like the order reverses (57.9 DQN, 56.4 PPO vs 22.5 and 26.3 static).

Online training (middle and right). Averaged over ρ and tested against the no-shift control (Holm over 24 contrasts), the oracle reward recovers the bot shift significantly ($+30.5$ DQN, $+33.2$ PPO DI; $p_{\text{holm}} < 10^{-6}$) and the score-only reward also does ($+27.6$, $+27.5$; $p_{\text{holm}} \leq 0.003$). The score-only reward cannot recover the human shift (PPO -8.7, $p_{\text{holm}} = 0.002$). No posterior-family reward gains significantly in either direction, and at $\rho = 1$ with covert bots online training with the posterior reward lets essentially every bot through (DI ≈ 0), *below* its own no-shift control: the reward inherits the scorer's error and teaches it to the policy.

Pilot in the v2 world. To check that these conclusions do not depend on the leave-after-pass assumption, we repeated the study in the abandonment-at-presentation world with a 7-

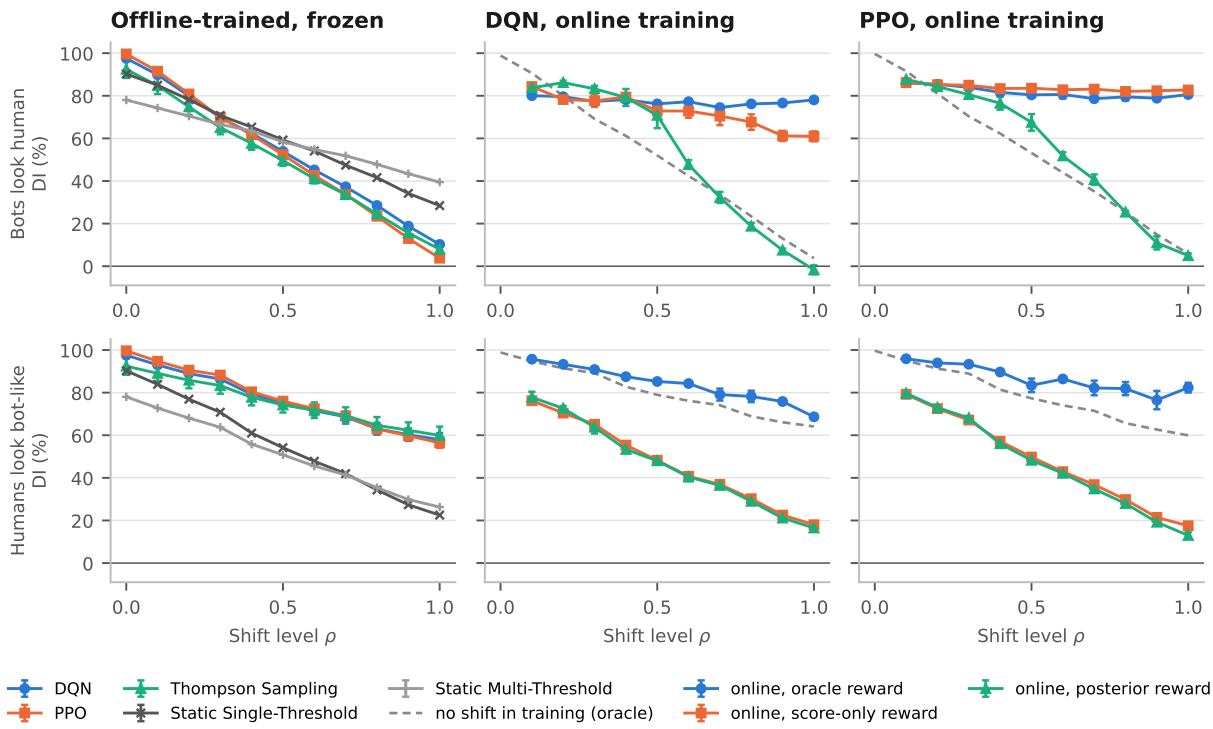


FIGURE 4. **DI on the held-out block as the scorer's calibration degrades (grounded world, mean ± SE over 5 training seeds). Left: offline-trained policies, frozen. Middle/right: online training through the shift with three rewards; dashed: oracle reward trained without the shift. Online points start at $\rho = 0.1$.**

TABLE 6. **v2-World Pilot at Full Shift ($\rho = 1$): DI, One Seed, Validation Block. At $\rho = 0$ Offline DI Is 100.0 (DQN), 99.8 (PPO), 84.5 (TS)**

	Bots look human			Humans look bot-like		
Training	DQN	PPO	TS	DQN	PPO	TS
Offline, frozen	8.9	7.6	−7.3	46.2	48.7	50.5
Online, oracle	69.6	54.3	61.9	61.4	70.4	75.0
Online, score-only	62.0	39.9	22.5	18.0	18.0	17.9
Online, posterior	0.3	5.0	3.5	17.2	13.3	16.7

feature state (adding the highest level solved and the number of steps passed). This pilot uses one seed and the validation block, so it is indicative only (Table 6). The pattern is the same: frozen policies collapse under covert bots, oracle online training recovers both directions, and the posterior reward collapses.

V. DISCUSSION AND LIMITATIONS

What the evidence supports. With a well-calibrated behavioural score, a learned controller separates humans from bots better than static thresholds on held-out recordings, the score is the component that matters most, and the advantage holds when challenge outcomes are stochastic, although the best learner then changes from DQN to Thompson Sampling.

What it does not. (1) *Deployment reward*: the policies need the true class during training. Rewards from observable outcomes fail, and scorer-based rewards are only as good as the scorer; a deployment would need periodic labelled data (e.g., confirmed abuse or manual review) for oracle-style updates and a monitor of the scorer's calibration. (2) *Usability is modelled, not measured*: pass rates, times and abandonment come from published studies of other populations, and no participant interacted with the 11 levels. (3) *Data*: 3 human participants and 4 bot generators; the evaluation block is small and near ceiling. (4) *Simulator*: no real CAPTCHA provider, no request-rate, CPU or latency model (hence no server-resource DoS claim), and no adaptive or VLM-based attacker, although the drift study shows what happens when unseen bots arrive. (5) *Post-hoc choice*: the drift-study scorer seed was chosen after seeing the external data.

Future work. A multi-seed run of the v2 world with the extended state and a label-free reward with a fixed bot-share prior is in progress. Beyond it: a user study of the challenge ladder, integration with real CAPTCHA providers, a queueing model for server load, and attackers that adapt to the learned policy.

VI. CONCLUSION

We framed CAPTCHA orchestration as learning a mapping from a calibrated behavioural score and session context to an 11-level challenge ladder, and evaluated it in a simulator grounded in published CAPTCHA user studies with

a leakage-controlled data protocol. On held-out recordings DQN reaches 98.9 DI ($\pm$0.4 over seeds) against 78.7 for the best static policy, and without the score it drops to 23.6; with stochastic challenge outcomes Thompson Sampling performs best. The same experiments show where the approach stops: no reward computed from observable outcomes alone trained a working policy, and policies trained or adapted with scorer-derived rewards inherit the scorer's miscalibration. A practical system therefore needs a calibrated scorer, a source of occasional ground-truth labels, and calibration monitoring; with those, RL is a viable replacement for hand-tuned score thresholds.

ACKNOWLEDGMENT

AI tools were used for language editing and formatting. The authors thank the participants who provided mouse-movement data.

REFERENCES

[1] Imperva, "2025 Imperva Bad Bot Report," Thales Group, 2025.
[2] M. C. Mazilu, "Web scraping and ethics in automated data collection," in *Smart Innovation, Systems and Technologies*, vol. 276. Springer, 2022.
[3] E. Ferrara, O. Varol, C. Davis, F. Menczer, and A. Flammini, "The rise of social bots," *Commun. ACM*, vol. 59, no. 7, pp. 96–104, 2016.
[4] M. H. N. Ba, J. Bennet, M. Gallagher, and S. Bhunia, "A case study of credential stuffing attack: Canva data breach," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, 2021.
[5] L. von Ahn, M. Blum, N. J. Hopper, and J. Langford, "CAPTCHA: Using hard AI problems for security," in *Advances in Cryptology – EUROCRYPT 2003*, pp. 294–311.
[6] M. Guerar, L. Verderame, M. Migliardi, F. Palmieri, and A. Merlo, "Gotta CAPTCHA 'em all: A survey of 20 years of the human-or-computer dilemma," *ACM Comput. Surv.*, vol. 54, no. 9, Art. 192, 2021.
[7] E. Bursztein, S. Bethard, C. Fabry, J. C. Mitchell, and D. Jurafsky, "How good are humans at solving CAPTCHAs? A large scale evaluation," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 399–413, doi: 10.1109/SP.2010.31.
[8] A. Searles, Y. Nakatsuka, E. Ozturk, A. Paverd, G. Tsudik, and A. Enkoji, "An empirical study & evaluation of modern CAPTCHAs," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 3081–3097.
[9] M. Qi *et al.*, "VIPER Strike: Defeating visual reasoning CAPTCHAs via structured vision-language inference," arXiv:2601.06461, 2026.
[10] Z. Wu *et al.*, "MCA-Bench: A multimodal benchmark for evaluating CAPTCHA robustness against VLM-based attacks," arXiv:2506.05982, 2025.
[11] A. Plesner, T. Vontobel, and R. Wattenhofer, "Breaking reCAPTCHAv2," in *Proc. IEEE COMPSAC*, 2024.
[12] Google reCAPTCHA. [Online]. Available: https://www.google.com/recaptcha/about
[13] M. Sepczuk and Z. Kotulski, "A new risk-based authentication management model oriented on user's experience," *Comput. Secur.*, vol. 73, pp. 17–33, 2018.
[14] T. S. Sethi, M. Kantardzic, L. Lyu, and J. Chen, "A dynamic-adversarial mining approach to the security of machine learning," *WIREs Data Mining Knowl. Discov.*, vol. 8, 2018.
[15] S. Sivakorn, J. Polakis, and A. D. Keromytis, "I'm not a human: Breaking the Google reCAPTCHA," in *Black Hat Asia*, 2016.
[16] W. A. Yousef, I. Traoré, and W. Briguglio, "Classifier calibration: With application to threat scores in cybersecurity," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 1994–2010, 2023.
[17] Google Cloud Armor. [Online]. Available: https://cloud.google.com/security/products/armor
[18] M. Umar and S. Ayyub, "A review on evolution of various CAPTCHA in the field of web security," *IJRASET*, 2022.
[19] J. Yan and A. S. El Ahmad, "Usability of CAPTCHAs or usability issues in CAPTCHA design," in *Proc. SOUPS*, 2008.
[20] E. Bursztein, J. Aigrain, A. Moscicki, and J. C. Mitchell, "The end is nigh: Generic solving of text-based CAPTCHAs," in *Proc. USENIX WOOT*, 2014.
[21] I. Akrout, A. Feriani, and M. Akrout, "Hacking Google reCAPTCHA v3 using reinforcement learning," arXiv:1903.01003, 2019.
[22] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*, PMLR vol. 139, pp. 8748–8763, 2021.
[23] S. Khan, C. Devlen, M. Manno, and D. Hou, "Mouse dynamics behavioral biometrics: A survey," *ACM Comput. Surv.*, vol. 56, no. 6, pp. 1–33, 2024, doi: 10.1145/3640311.
[24] C. Shen, Z. Cai, X. Guan, and J. Wang, "Mouse dynamics for continuous user authentication: An experimental analysis," in *Proc. NSS*, Springer, 2013, pp. 250–261.
[25] M. Antal and E. Egyed-Zsigmond, "Intrusion detection using mouse dynamics," *IET Biometrics*, vol. 8, no. 5, pp. 285–294, 2019.
[26] M. Takeda, T. Sato, H. Saito, H. Iwasaki, I. Nambu, and Y. Wada, "Explanation of Fitts' law in reaching movement based on human arm dynamics," *Sci. Rep.*, vol. 9, Art. 19804, 2019.
[27] S. Adarsh, P. V. V. Sankaranarayanan, and P. A. Suresh, "Emulating human-like mouse movement using Bézier curves and behavioural models for advanced web automation," *IJIRT*, vol. 12, no. 1, 2025.
[28] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez, "BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection," *Pattern Recognit.*, vol. 127, 2022.
[29] M. Antal, K. Buza, and N. Fejer, "SapiAgent: A bot based on deep learning to generate human-like mouse trajectories," *IEEE Access*, vol. 9, 2021.
[30] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, "Detection of advanced web bots by combining web logs with mouse behavioural biometrics," *Digit. Threats: Res. Pract.*, vol. 2, no. 3, Art. 24, 2021, doi: 10.1145/3447815.
[31] A. Acien, A. Morales, J. Fierrez, R. Vera-Rodriguez, and O. Delgado-Mohatar, "BeCAPTCHA: Behavioral bot detection using touchscreen and mobile sensors benchmarked on HuMIdb," *Eng. Appl. Artif. Intell.*, vol. 98, 2021.
[32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
[33] D. Wei *et al.*, "Predicting mouse click position using long short-term memory model trained by joint loss function," in *Proc. CHI EA*, 2021.
[34] R. Sahay *et al.*, "Aura-CAPTCHA: A reinforcement learning and GAN-enhanced multi-modal CAPTCHA system," arXiv:2508.14976, 2025.
[35] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 661–670, doi: 10.1145/1772690.1772758.
[36] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," *Found. Trends Mach. Learn.*, vol. 11, no. 1, pp. 1–96, 2018, doi: 10.1561/2200000070.
[37] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
[38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
[39] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
[40] Y. Cui, X. Han, J. Chen, X. Zhang, J. Yang, and X. Zhang, "FraudGNN-RL: A graph neural network with reinforcement learning for adaptive financial fraud detection," *IEEE Open J. Comput. Soc.*, 2025.
[41] C. Huang, Y. Wu, Y. Zuo, K. Pei, and G. Min, "Towards experienced anomaly detector through reinforcement learning," in *Proc. AAAI*, 2018.
[42] P. K. Mvula, P. Branco, G.-V. Jourdan, and H. L. Viktor, "A systematic literature review of cyber-security data repositories and performance assessment metrics for semi-supervised learning," *Discover Data*, 2023.
[43] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.
[44] P. Christen, D. J. Hand, and N. Kirielle, "A review of the F-measure: Its history, properties, criticism, and alternatives," *ACM Comput. Surv.*, vol. 56, no. 3, Art. 73, 2023.

[45] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
[46] Á. Fülöp, L. Kovács, T. Kurics, and E. Windhager-Pokol, Balabit Mouse Dynamics Challenge data set, 2016. [Online]. Available: https://github.com/balabit/Mouse-Dynamics-Challenge